Failure Prediction in Cloud Native 5G Core With eBPF-based Observability

Junichi Kawasaki*†, Daiki Koyama*, Takuya Miyasaka*†, Tomohiro Otani†
*KDDI Research Inc., Saitama, Japan
†KDDI Corporation, Tokyo, Japan
{ju-kawasaki, da-koyama, ta-miyasaka, tm-otani}@kddi.com

2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring) | 979-8-3503-1114-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/VTC2023-Spring57618.2023.10200028

***Abstract*— Today, the advances in virtualization technologies are dramatically changing network architecture as well as operation style. In particular, cloud-native network functions (CNFs) provide scalability, flexibility, and lightness to network services, which can be leveraged to satisfy vehicle application requirements in the 5G and beyond 5G era. However, their microservice architecture causes a new challenge in the operation of cloud-native networks due to the larger number of compositions and dynamic topology changes. To address this situation, artificial intelligence (AI)/machine learning (ML) technologies are expected to support operational tasks. One of these tasks is failure prediction that is a key enabler for proactive network operation to minimize the failure impact. For reliable failure prediction, not only ML methods but also data used for ML training should be prepared with a fine granularity. In this paper, we utilize the extended berkeley packet filter (eBPF) to collect fine-grained information from the virtual infrastructure where CNFs are deployed. By training a long short-term memory (LSTM) model on the collected data, we develop the failure prediction model that can provide the future transition of key performance indicators (KPIs) in a cloud-native 5G core network (5GC). Our experiment on the lab environment shows that the prediction model trained on eBPF data outperforms other models trained without them.**

Keywords—eBPF, observability, failure prediction, predictive detection, CNF

I. INTRODUCTION

With the widespread adoption of 5G, the use of mobile networks in vehicles is accelerating. In the future, cutting-edge applications such as automated and remote driving are expected to be realized with the 5G and beyond 5G networks. Therefore, system availability is essential in such a mobile network, i.e., a critical communication infrastructure for supporting the future transportation. To improve system availability, network operators need to detect and mitigate network failures immediately.

On the other hand, virtualization in the network industry, i.e., network function virtualization (NFV), has driven a significant shift in network architecture. With the split of software and hardware, network operators are able to integrate their network functions (NFs) on virtual machines as virtual network functions (VNFs) or containers as cloud-native network functions (CNFs) according to the customer requirements and infrastructure capacity. In the 5G and subsequent beyond 5G era, mobile networks are expected to be utilized not only by traditional devices such as smartphones but also in various use cases such as connected cars, which requires the flexibility and scalability of NFV. To meet such requirements, the third generation partnership project (3GPP) adopts CNF-friendly network architecture called service-based architecture (SBA) from its stand-alone 5G core network (5GC) specification [1]. Therefore, we expect network operators will deploy more NFs of 5GC as CNFs.

In the actual network operation, dealing with network failures is one of the most critical factors. There are two types of network failures: network failures that affect service quality immediately (e.g., network link down due to fiber-cut) and network failures that take a certain amount of time to affect service quality (e.g., gradual packet loss increase due to the optical transceiver's deterioration). While the former type of failure can only be handled after it occurs, the latter type can be dealt with before it causes service impact by predicting network failures in advance. This type of proactive-style network operation is required to maintain carrier-grade system availability.

However, with the widespread of CNFs in network services, including 5G, the number of NFs becomes significantly huge, and the connection among NFs also becomes complex, which makes such proactive operation even more challenging. To struggle with this challenge, many efforts have been made in the research area of network operation with artificial intelligence (AI) and machine learning (ML) called AIOps [2]-[4]. In particular, the prediction of network failures, which is a key enabler for proactive operation, is expected to be realized by AIOps. To achieve reliable prediction of network failures, detailed statistical data about the network play a crucial role in training ML models. In CNFs, *Observability* is an essential element for measuring detailed system behavior in the complex virtual infrastructure. The extended berkeley packet filter (eBPF), a function introduced in the Linux kernel, is one of the technologies to realize such observability. eBPF can collect process-level fine-grained data in a kernel space, such as CPU queue latency and TCP retransmission count.

In this paper, we propose a novel failure prediction method in cloud-native 5GC using eBPF. In our method, lots of fine-grained statistical data obtained by eBPF from the CNF infrastructure are leveraged. We train a long short-term memory (LSTM) model on these data to predict 5G key performance indicators (KPIs) such as the number of user equipment (UE) registration failures. In our evaluation, we measure the performance of failure prediction models in two types of network failure events (packet loss and CPU overload in 5GC). The evaluation results of the former failure type show that the model trained on the statistical data collected by eBPF outperforms the models trained without eBPF data in terms of F1-score.

To summarize, the main contributions of this paper are:

979-8-3503-1114-3/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:52:49 UTC from IEEE Xplore. Restrictions apply.

- We propose a novel failure prediction method to predict network failures in 5GC CNFs by the LSTM model trained on fine-grained eBPF data.
- We evaluate the performance of failure prediction in 5GC CNFs and confirm the effectiveness of eBPF data. Furthermore, we analyze which types of data among many eBPF metrics are effective in failure prediction.

The remainder of this paper is organized as follows. In Section II, we review related work. In Section III, we introduce our proposed architecture consisting of data collection and failure prediction. Then, we present the results of an experimental evaluation in 5GC CNFs in Section IV, and conclude the paper in Section V.

II. Related Work

We review the existing studies on eBPF-based observability, LSTM-based failure analysis, and KPI prediction in this section.

eBPF-based observability eBPF was originally developed for packet filtering in the Linux kernel, and has been extended to support other functionalities such as monitoring and packet processing. The general workflow of monitoring is as follows. The BPF bytecode created by a user is loaded into a kernel space. After verification, the eBPF tools run in a kernel to collect data. There are two types of data output from a kernel to a user space, synchronous and asynchronous. In the synchronous mode, an event is sent to a user space when an event is detected in a kernel. In the asynchronous mode, the data collected in a kernel are temporarily stored in a memory, and provided to an user as statistics when a request is received. eBPF can provide process-level data from a kernel with low overhead [5], which enables scalable monitoring. In addition, it does not require any additional change in kernel nor application codes while just leveraging the functionalities provided by the Linux kernel version 3.15+.

Several works have been presented that use eBPF to support observability. [5] developed the analysis system for enabling container network observability, where 22 eBPF tools were implemented to collect multiple metrics related to TCP, UDP, and HTTP. The experiment on test applications showed that the metrics such as connection numbers varied when network failures were injected. [6] implemented the performance monitoring system using eBPF. It was shown that this system can analyze function calls on processors and HTTP traffic on the connectors of microservice applications for the payment system. [7] applied eBPF to the SD-WAN environment to show the network resiliency based on the infrastructure statistics. In the demonstration, the number of TCP retransmissions was collected periodically. When one of the WAN tunnels was saturated, the retransmission number increased, which triggered the controller to switch the tunnel to a redundant one. [8] proposed ViperProbe, an eBPF-based microservice metrics collection framework. From a variety of metrics collected from the infrastructure of microservices, critical metrics such as sentbytes were extracted by k-shape clustering. The experiment on test applications showed that the number of auto-scaling performed according to the values of critical metrics was much lower than that performed based on the existing Kubernetes configuration. Although [5]-[8] presented the effectiveness of eBPF in enhancing microservice observability, they analyzed the original metric values by some threshold for monitoring and did not analyze the data collected by eBPF to derive more useful insights such as failure prediction.

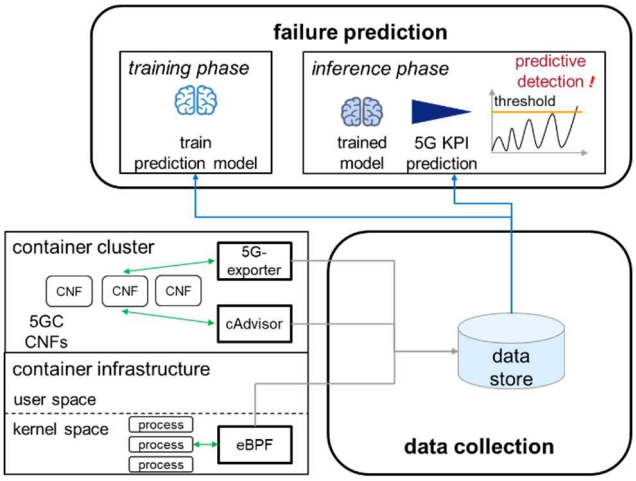


Figure 1: Overall Architecture

LSTM-based failure analysis One of the major research areas in AIOps is network failure analysis. From the many types of ML algorithms, LSTM is widely utilized in anomaly detection and failure prediction for various environments. For example, the traffic data obtained from SDN [9] and IP networks [10], the various data collected by SNMP from enterprise computer networks [11], and the disk data from data center servers [12] were used to train LSTM models respectively, and the evaluation results showed good performance in detecting failures. In addition, [13] compared the performance of five algorithms for anomaly detection, where LSTM achieved the highest score in the experiment on the applications on Kubernetes.

KPI prediction The prediction of KPIs is one of the significant research topics for service assurance. [14] and [15] evaluated KPI anomaly detection using ML. The number of page views in [14] and several KPIs such as active users in [15] were predicted by the ML models, which presented high performance. However, these papers focused on KPIs in the application layer, not network. [16] set four KPIs in mobile networks, handover success rate (HOSR), downlink throughput, downlink traffic, and RRC connection request rate, and the Ada-Boost model was created to predict the future transition of these KPIs. However, this model handled only the KPI values themselves for the model input, and the use of other metrics collected from the network was out of scope.

III. Architecture and Implementation

In this section, we describe the architecture of our proposal and its implementation. The proposed architecture is presented in III-A, and two functional blocks in the architecture, data collection and failure prediction, are explained in III-B and III-C respectively.

A. Architecture Overview

Our approach to predict future potential failures is shown in Figure 1. Failure prediction in this paper is performed based on 5G KPIs. A future potential failure is detected when the prediction of future KPI values does not satisfy service requirements. The overall architecture consists of two functional blocks, data collection and failure prediction. The

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:52:49 UTC from IEEE Xplore. Restrictions apply.

Table 1: Implemented eBPF Tools

Tool	Description
execsnoop	Trace new processes.
runqlat	Run queue (scheduler) latency.
runqlen	Summarize scheduler run queue length.
funccount	Count functions and tracepoints.
tcpaccept	Trace TCP accepts.
tcpconnlat	Trace TCP active connection latency.
tcpdrop	Trace TCP kernel-dropped packets.
tcplife	Trace the lifespan of TCP sessions.
tcpretrans	Trace or count TCP retransmits.
tcprtt	Summarize TCP round trip time.
tcpstates	Trace the TCP session state changes.
tcptracer	Trace TCP connections.
tcpwin	Trace TCP congestion window parameters.
solisten	Trace socket listen events.

data collection obtains three types of data, fine-grained information collected by eBPF, basic information by cAdvisor [17], and 5G information by *5G-exporter*. They are collected from 5GC CNFs and their container infrastructure. The collected data are accumulated in the data store. The failure prediction function develops prediction models using the stored data sets in the training phase. In the inference phase, the trained models output the future transition of metrics including 5G KPIs, which enables early predictive detection of KPI violation.

B. Data Collection

In the data collection, there are three types of collection, eBPF, cAdvisor, and 5G-exporter as shown in Figure 1.

From the numerous eBPF tools, we adopt 14 tools listed in Table 1 for the eBPF data collection considering our target failure scenarios, packet loss and CPU overload in 5GC, which are described in IV-B in detail. Here, *execsnoop*, *runqlat*, *runqlen*, and *funccount* are selected for the CPU overload case. In this case, where CPU resources are saturated, these tools that collect detailed information related to processes such as run queue latency are expected to derive effective metrics. Other tools, TCP-related types and *solisten*, are implemented for the packet loss case. This failure is injected in the control plane of 5GC, where the TCP protocol is used for transport layer. Thus, these tools that gather detailed information about TCP connections such as retransmission count may provide useful metrics. In addition, we implement a feature of CNF correlation to specify source CNFs of metrics that are not provided in the raw data collected by eBPF. With the mapping table created using the information of process ID and IP address for correlation, the eBPF data collection can obtain eBPF metrics with source information.

We also utilize cAdvisor, an open-source tool that can be introduced on a Kubernetes environment, to obtain basic metrics of running containers such as CPU usage and traffic volume. In contrast to eBPF, this tool provides metrics including the information about where the metrics come from.

Furthermore, we implement 5G-exporter to derive 5G specific metrics from the logs of 5GC CNFs. In 5G mobile networks, general procedures specified in 3GPP such as registration run for UE communication [18]. Based on these procedures, a variety of 5G metrics are defined [19], [20], which are important factors for network operators to track the

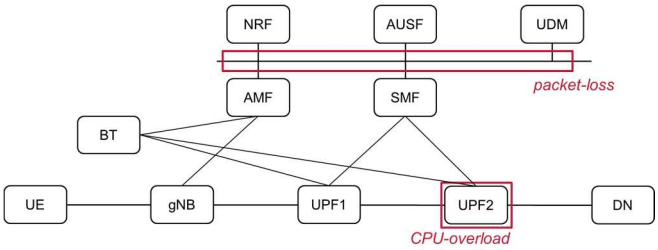


Figure 2: 5GC CNFs on Testbed

quality of 5G network services. For collecting these 5G information, the 5G-exporter implemented in the container cluster traces log messages of 5GC CNFs. It counts up specific logs related to registration, deregistration, PDU session creation, and PDU session release to provide the number of requests and successes for these procedures. In addition, the number of failures in each procedure is calculated from the number of requests and successes to be added as 5G-exporter metric.

C. Failure Prediction

The failure prediction leverages the data collected by eBPF, cAdvisor, and 5G-exporter to provide a predicted future transition of metrics including 5G KPIs. The data accumulated in the data store are extremely high dimensional data, which are too large and complicated for humans or conventional approaches to analyze. Therefore, we adopt ML technologies to derive failure prediction from this big data. Here, we use LSTM in this paper, one of the major deep learning algorithms as described in II, which can analyze multi-variant time-series data to provide future prediction. Since the main scope of our proposal is eBPF-based observability, we focus on one ML method in this paper and leave other ones for future work.

In our implementation, we design a failure prediction model using LSTM that receives part of the metrics collected by eBPF, cAdvisor, and 5G-exporter and provides a prediction on the future transition of metrics including 5G KPIs such as the number of UE registration failures and throughput. For predictive detection, a certain threshold is set, and the alarm of a future potential failure is raised when a prediction value of, for example, the number of UE registration failures at a future point of time exceeds the pre-defined threshold.

IV. Experiment and Results

We performed an experiment to evaluate our proposal, failure prediction with eBPF-based observability. First, we prepared a test environment. Next, we ran test scenarios iteratively on the testbed while collecting data by eBPF, cAdvisor, and 5G-exporter. Finally, we trained and evaluated failure prediction models. Here, we evaluated three types of models that were trained on the eBPF, cAdvisor, and 5G-exporter data respectively to explore the effectiveness of eBPF data for failure prediction. In addition, we also evaluated several types of models developed with feature selection to study which metrics of eBPF data are effective. We describe the details in the following subsections.

A. Testbed

We built a test environment in our lab using a physical server with Linux kernel version 5.4, two Intel Xeon CPUs Gold 6142 2.60GHz, and 256GB memory. A set of 5GC CNFs (AUSF, UDM, SMF, AMF, NRF, UE, gNB, two UPFs, and DN) was deployed by Kubernetes version 1.22.2 on this server

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:52:49 UTC from IEEE Xplore. Restrictions apply.

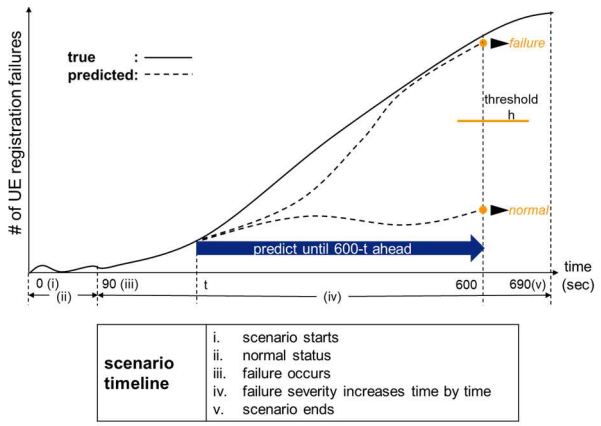


Figure 3: Scenario and Prediction Label in *packet-loss*

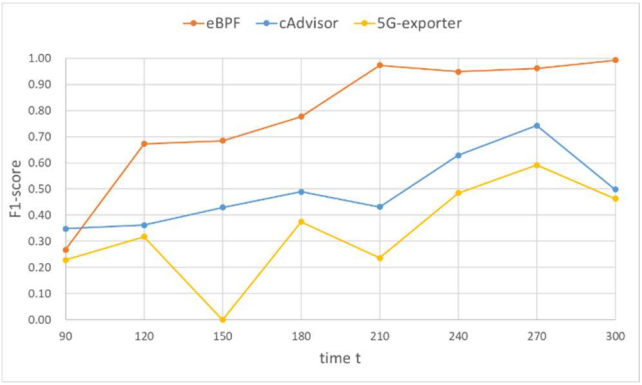


Figure 4: Results of Data Collection Type in *packet-loss*

Table 2: Hyper-parameter of LSTM

Hyper-parameter	Value
Number of epochs	50
Learning rate	0.001
Batch size	64
Optimizer	Adam
Hidden activation	Relu
Output activation	Linear

using Open5Gcore release 6 [21] as shown in Figure 2. In addition, the Benchmarking Tool (BT), UE/gNB simulator provided in the Open5Gcore, was also created on the same server to generate signaling messages for registration and deregistration. When a request is sent by UE, respective procedure runs in 5GC CNFs on the testbed according to the specification in [18]. After a registration request from UE is received on AMF, authentication is processed on AUSF and UDM. Then, SMF establishes a PDU session between UE, gNB, and UPF. In the deregistration, after receiving a request from UE, AMF requests SMF to release the established PDU session. We also built the data collection function using eBPF, cAdvisor, and 5G-exporter. Here, we utilized the BCC library [22], one of the major libraries for introducing eBPF.

B. *Scenario and Data Set*

To evaluate failure prediction models, failure scenarios are required where some failure event occurs and its severity gradually gets worse. We set two such scenarios, *packet-loss* and *CPU-overload*. In the *packet-loss* case, packet loss is injected on the bridge of the control plane in 5GC as shown in Figure 2, and the loss rate is linearly increased from 0% upto 5% in 600 seconds. In the *CPU-overload*. case, CPU overload is injected on UPF2, and the CPU usage is linearly increased from 0% to 100% in 600 seconds. While running these scenarios, BT repeats the constant registration and deregistration of UEs to simulate regular signaling traffic. In addition, user data traffic is also generated between UE and DN using iPerf [23]. The timeline of each scenario is described in Figure 3.

We repeated step i – v for both *packet-loss* and *CPU-overload* to generate data sets of failure cases. Furthermore, we also generated data sets in a normal case by not injecting any failure in step iii and iv. While running the scenarios on the testbed, the data collection gathered all types of data by eBPF, cAdvisor, and 5G-exporter every 10 seconds, providing data sets for evaluation. The data sets obtained in the first period including 150 samples for each failure scenario and 450 normal samples were used for training, and those in the second period containing 75 samples for each failure scenario and 225 normal samples were used for test. Each data set consisted of a total of 3326 metrics, including 2262 metrics originated from eBPF data collection, 1048 from cAdvisor, and 16 from 5G-exporter.

C. *Target KPI and Evaluation Criteria*

We explain how to measure the performance of failure prediction models. First of all, a target metric and time for prediction is necessary for evaluation. In our evaluation, the number of UE registration failures, one of the significant KPIs in mobile networks, was set as a prediction target for *packet-loss*, and downstream throughput on UPF2, which is also a crucial KPI, was set for *CPU-overload*. For a prediction target time, we set 600 seconds after a scenario starts.

For the first step in the evaluation, the failure prediction model was created on training data sets from t-60 until t-10 to predict the transition of respective target KPI at prediction time t. Here, the predicted transition is from 10 until 600-t seconds ahead. After training, the model inputs time-series test data sets from t-60 until t-10 and outputs the future transition of KPI until 600-t ahead. Then, the prediction label on future network status, *normal* or *failure*, was determined based on the predicted KPI at 600-t seconds ahead by threshold h as shown in Figure 3. If the predicted value was larger than h for *packet-loss* or smaller than h in the case of *CPU-overload*, the prediction label was *failure*. On the other hand, if the value was smaller than h for the *packet-loss* or larger than h for *CPU-overload*, its prediction was *normal*. In our evaluation, we set h according to the three sigma rule that is widely adopted in anomaly detection. In this rule, h was calculated with the mean $\pm$ three standard deviations of normal samples. Finally, by comparing a prediction label provided by the trained model with an answer label of test data sets, *normal* for normal samples and *failure* for *packet-loss* and *CPU-overload* samples, a confusion matrix was created, providing precision, recall, and F1-score.

Furthermore, one of the key aspects of predictive detection is how early the failure prediction models can capture a sign of failure and raise the alarm while an failure level is increasing step by step. Hence, we changed the prediction time t in our experiment to explore the model performance of each timing.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:52:49 UTC from IEEE Xplore. Restrictions apply.

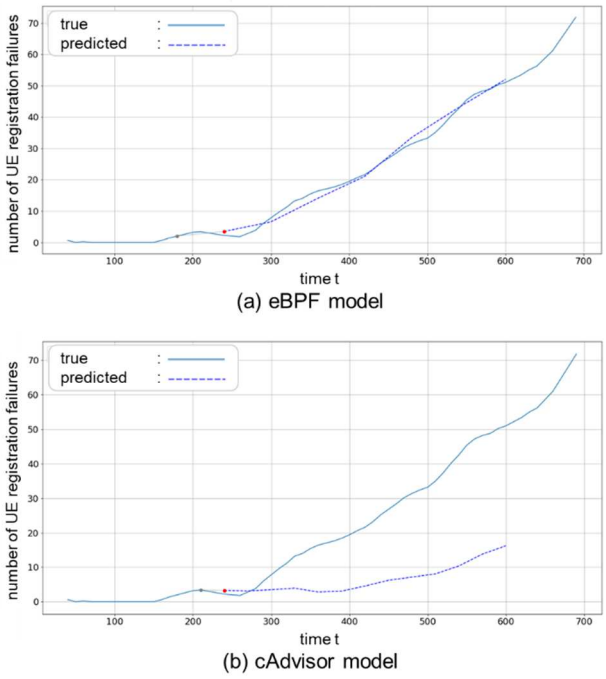


Figure 5: Prediction Examples at $t = 210$ in *packet-loss*

D. Evaluation of Data Collection Type

To study the effectiveness of eBPF data on improving the performance of failure prediction models, we evaluated the performance of three prediction models that were trained on the data collected by eBPF, cAdvisor, and 5G-exporter, called the eBPF model, cAdvisor model, and 5G-exporter model respectively. These models were developed using LSTM with the hyper-parameters shown in Table 2 to predict the future transition of KPI. We changed the prediction time t every 30 seconds between 90 and 300 for each model and evaluated precision, recall, and F1-score for two failure scenarios.

Figure 4 shows the results of the F1-score at respective time t for each model created for *packet-loss*. At $t = 90$, when the increase of the packet loss rate has just started, all F1-scores are under 0.4. The performance of the eBPF model is improving as t is large, and this model can predict future increase in registration failures with an F1–score exceeding 0.9 at $t = 210$. On the other hand, the cAdvisor model and 5G-exporter model, which was trained without eBPF data, cannot detect future increase with F1-score > 0.9 even at $t = 300$, though their performance increases at a certain level at later time. For further comparison, prediction examples at $t = 210$ are presented in Figure 5. The graph (a) is provided by the eBPF model and (b) is provided by the cAdvisor model. The eBPF model predicted future transition of the number of UE registration failures with high accuracy while the cAdvisor model did not predict as accurately as such. On the other hand, Figure 6 shows the evaluation results for *CPU-overload*. In this case, the eBPF model and the cAdvisor model achieved F1-score > 0.9 in predicting future decrease in throughput at $t = 150$ and 180 respectively, while no performance enhancement was observed in the 5G-exporter model.

From the above evaluation results, the eBPF model presented the highest score for *packet-loss*, which means that the fine-grained statistical data obtained by eBPF tools highly contributed to the model performance in this type of failure. It showed that our hypothesis described in III-B was valid. eBPF

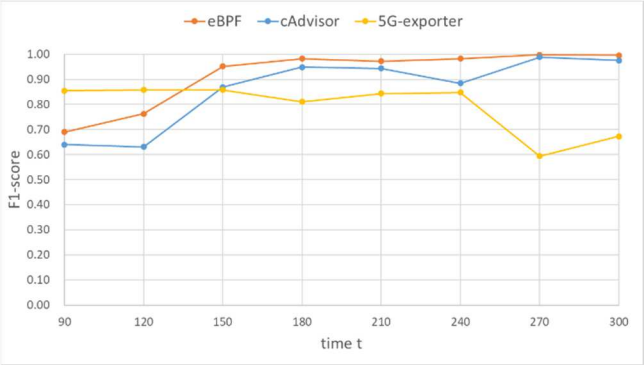


Figure 6: Results of Data Collection Type in *CPU-overload*

data include a variety of metrics related to TCP connections as shown in Table 1, while neither cAdvisor nor 5G-exporter data contain such information. This *packet-loss* case causes a network failure on the 5GC control plane where the TCP protocol is used, thus, eBPF model trained on detailed information about TCP connections outperforms other models in predicting future failure increase, or KPI violation. On the other hand, in the case of *CPU-overload*, the models except for the 5G-exporter model showed good performance in failure prediction, and the contribution of eBPF to the model performance was limited compared with the previous case. This result also comes from the types of metrics that each data collection gathers. While eBPF data contains the details of CPU processes, cAdvisor also collects basic information about CPU such as usage. Both CPU-related metrics had a correlation with this *CPU-overload* scenario, thus, the eBPF and cAdvisor model presented good results while the 5G-exporter model that does not use any CPU-related metric showed lower performance.

E. Evaluation of Feature Selection

We performed an further experiment for *packet-loss*, where the effectiveness of eBPF was clearly observed, to explore which metrics in lots of eBPF data are crucial for failure prediction. Here, each metric obtained by data collection was used as the feature in ML training. Thus, we changed the feature sets by feature selection in this experiment. The same data sets were used in training LSTM models with the same hyper-parameters in the previous experiment. We set $t = 210$, when the eBPF model achieved F1-score > 0.9. The feature selection was performed based on TCP-related tools in Table 1 and total 11 feature sets were applied for evaluation including the case of selecting all eBPF features as *all eBPF* as shown on Table 3. For example, in the case of *tcpaccept*, only the metrics collected by the *tcpaccept* tool were utilized for developing a failure prediction model. By evaluating each trained model with test data sets, precision, recall, and F1-score were obtained as shown in Table 3. It is observed that the *tcpretrans*, *tcprtt*, and *tcpwin* model show high performance in failure prediction. To clarify the effectiveness of these feature sets, we performed an additional evaluation of the model trained on eBPF data without the metrics collected by the *tcpretrans*, *tcprtt*, and *tcpwin* tools shown on the bottom row in the table. The performance of this model is much lower than that of the *all eBPF* model on the top row. This result shows that the metrics derived by these three tools take an essential role in developing models for *packet-loss*. This comes from the feature of packet loss in 5GC control plane. If packets are dropped in the control plane, the TCP protocol performs congestion control as well as retransmission of the

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:52:49 UTC from IEEE Xplore. Restrictions apply.

Table 3: Results of Feature Selection based on TCP-related Tools

Feature Sets Selected based on Tools	#Feat.	Precision	Recall	F1-score
all eBPF	2262	1.000	0.947	0.973
tcpaccept	2	0.266	0.933	0.414
tcpconnlat	10	0.287	0.960	0.442
tcpdrop	16	0.521	1.000	0.685
tcplife	156	0.256	0.973	0.406
tcpretrans	66	0.962	1.000	0.980
tcprtt	369	0.926	1.000	0.962
tcpstates	355	0.289	0.933	0.442
tcptracer	22	0.318	0.933	0.475
tcpwin	955	0.708	1.000	0.829
without tcpretrans, tcprtt, tcpwin	872	0.421	0.960	0.585

packets. There are many TCP connections in 5GC control plane handling a plenty of procedures, thus, this failure event causes many retransmissions, higher latency due to retransmit packets, and changes in the parameters of congestion control such as window size. Therefore, *tcpretrans*, *tcprtt*, and *tcpwin*, are significant tools in creating failure prediction models in this type of failure.

V. CONCLUSION

Failure prediction is expected to help network operators minimize service impact of failures that occur in cloud-native networks such as 5GC CNFs. Here, observability, fine-grained data collection, plays a key role in creating reliable failure prediction models. In this paper, we leveraged eBPF to collect process-level statistical data and built the model for predicting future KPI transition. On the experimental 5GC CNFs, we ran two failure scenarios, *packet-loss* and *CPU-overload*, while collecting data by eBPF, cAdvisor, and 5G-exporter. We developed three models using respective data collection type and performed comparative evaluation on the failure prediction performance. The results showed that the eBPF model outperformed other models for *packet-loss*, which means that eBPF data contributed to developing reliable models. To clarify which metrics highly contributed, we further measured the model performance using feature selection. The results showed that the eBPF tools that collect the metrics related to TCP retransmission, latency, and congestion control were effective, which suggests that the observability of these information need to be considered in applying failure prediction models to the cloud-native 5GC. For future work, we highlight the following directions to explore the applicability in real environments: (i) evaluate the monitoring overhead of eBPF; (ii) evaluate the prediction performance on larger networks; and (iii) extend our approach to more complex scenarios such as long-term degradation.

ACKNOWLEDGMENT

This work was conducted as part of the project entitled "Research and development of infrastructure technologies for innovative virtualization network (JPMI00316)" supported by the Ministry of Internal Affairs and Communications, Japan.

REFERENCES

[1] 3GPP, "System architecture for the 5G System," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 23.501, Sep. 2022, v. 17.6.0.

[2] Y. Dang, Q. Lin, and P. Huang, "Aiops: real-world challenges and research innovations," in 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), IEEE, May 2019, pp. 4–5.

[3] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, "Machine learning for anomaly detection: A systematic review," IEEE Access, vol. 9, pp. 78658–78700, May 2021.

[4] S. Wang, J. Balarezo, S. Kandeepan, A. Al-hourani, K. Gomez, and B. Rubinstein, "Machine Learning in Network Anomaly Detection: A Survey," IEEE Access, vol. 4, pp. 152379-152396, Nov. 2021.

[5] C. Liu, Z. Cai, B. Wang, Z. Tang, and J. Liu, "A protocol-independent container network observability analysis system based on eBPF," in Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst. (ICPADS), Dec. 2020, pp. 697–702.

[6] C. Cassagnes, L. Trestioreanu, C. Joly, and R. State, "The rise of eBPF for non-intrusive performance monitoring," in Proc. IEEE/IFIP Netw. Operations Manage. Symp. (NOMS), Apr. 2020, pp. 1–7.

[7] S. Troia, M. Mazzara, L. M. M. Zorello, and A. Pattavina, "Resiliency in SD-WAN with eBPF Monitoring: Municipal Network and Video Streaming Use Cases," in Proc. 17th Int. Conf. on the Design of Reliable Commun. Netw. (DRCN), Apr. 2021, pp. 1-3.

[8] J. Levin and T. A. Benson, "ViperProbe: Rethinking microservice observability with eBPF," in Proc. IEEE 9th Int. Conf. Cloud Netw. (CloudNet), Nov. 2020, pp. 1–8.

[9] M. Abdallah, N. A. L. Khac, H. Jahromi, and A. D. Jurcut, "A hybrid CNN-LSTM based approach for anomaly detection systems in SDNs," in Proc. 16th Int. Conf. Availability, Reliab. and Secur. (ARES), ACM, Aug. 2021, pp. 1–7.

[10] G. Qin, Y. Chen, and Y. Lin, "Anomaly detection using LSTM in IP networks," in Proc. 6th International Conference on Advanced Cloud and Big Data (CBD), Aug. 2018, pp. 334–337.

[11] R. Nair, C. P. Kasula, S. Vankayala, and N. Chakraborty, "IP network anomaly detection using machine learning," in Proc. IEEE 5th Int. Conf. for Convergence in Technology (I2CT), Mar. 2019, pp. 1–7.

[12] H. Yang, Z. Li, H. Qiang, Z. Li, Y. Tu, and Y. Yang, "ZTE-predictor: disk failure prediction system based on LSTM," in Proc. 50th Annual IEEE-IFIP Int. Conf. on Dependable Systems and Netw. - Supplemental Volume (DSN-S), Jun. 2020, pp. 17-20.

[13] Y. Matsuo and D. Ikegami, "Performance analysis of anomaly detection methods for application system on kubernetes with auto-scaling and self-healing," in Proc. 17th Int. Conf. on Netw. and Service Manage. (CNSM), Oct. 2021, pp. 464-472.

[14] J. Shi, G. He, and X. Liu, "Anomaly detection for key performance indicators through machine learning," in Proc. Int. Conf. Netw. Infrastruct. Digit. Content (IC-NIDC), Aug. 2018, pp. 1–5.

[15] H. Ren, B. Xu, Y. Wang, C. Yi, C. Huang, X. Kou, T. Xing, M. Yang, J. Tong, and Q. Zhang, "Time-series anomaly detection service at microsoft," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD), Aug. 2019, pp. 3009–3017.

[16] J. Moysen, F. Ahmed, M. Garcia-Lozano, and J. Niemela, "Big data-driven automated anomaly detection and performance forecasting in mobile networks," in Proc. IEEE Globecom Workshops, Dec. 2020, pp. 1–5.

[17] cAdvisor, https://github.com/google/cadvisor

[18] 3GPP, "Procedures for the 5G System," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 23.502, Sep. 2022, v. 17.6.0.

[19] 3GPP, "5G performance measurements," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 28.552, Sep. 2022, v. 17.8.0.

[20] 3GPP, "5G end to end Key Performance Indicators (KPI)," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 28.554, Sep. 2022, v. 17.8.0.

[21] Open5GCore, https://www.open5gcore.org/

[22] BCC, https://github.com/iovisor/bcc

[23] Iperf, https://iperf.fr

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:52:49 UTC from IEEE Xplore. Restrictions apply.